Тема 4. Програмиране на задачи с разклонение (избор). Логически операции и сравнения. Оператори if и switch. Структуриране на програми с меню.

Цел: Съставяне, тестване и анализ на C++ програми с условно изпълнение, множествен избор и проверка на входни данни.

Очаквани резултати

- Да съставяте условия с операторите за сравнение и логическите операции !, && и ||.
- Да използвате if, if...else и else if за избор между алтернативи.
- Да използвате switch, case, break и default за избор по конкретна стойност.
- Да проверявате входните данни и граничните случаи и да изграждате меню за избор.

1. Логически изрази и операции за сравнение

Логическият израз има резултат true (истина) или false (лъжа). В C++ условие с числова стойност 0 е false, а с ненулева стойност — true; при работа с условия е за предпочитане типът bool.

Операции за сравнение:

- < — по-малко; <= — по-малко или равно; > — по-голямо; >= — по-голямо или равно.
- == — равно; != — различно.

Логически операции:

- !условие — отрицание (NOT); обръща логическата стойност.
- условие1 && условие2 — логическо И (AND); true само ако и двете условия са true.
- условие1 || условие2 — логическо ИЛИ (OR); true, ако поне едно условие е true.

> Важно: „=“ присвоява стойност, а „==“ сравнява две стойности. При && и || C++ прилага кратка оценка: втората част не се пресмята, ако резултатът вече е известен.

Пример: a = 5, b = 15, c = 80.

Израз	Резултат	Обяснение
a < b	true	5 < 15
b == 3 * a	true	15 == 15
a > b	false	5 не е по-голямо от 15
(a + b < 20) && (b / a == 3)	false	20 < 20 е false
(a + b < 12 && b / a == 3) || c == 80	true	c == 80 е true

Приоритет от по-висок към по-нисък: скоби, унарни операции (! и унарен минус), *, /, %, +, −, <, <=, >, >=, ==, !=, &&, ||. За четимост използвайте скоби при комбинирани условия.

```
bool validMonth = month >= 1 && month <= 12;
bool isWinter = month == 12 || month == 1 || month == 2;
bool notWinter = !isWinter;
```

2. Оператор if: избор при изпълнено условие

Операторът if изпълнява действие само когато условието е true. Ако са нужни няколко действия, ги поставете в блок с фигурни скоби.

```
if (условие) {
   // действия при вярно условие
}
```

Пример 1. Подреждане на две числа

Потребителят въвежда две цели числа a и b. Ако a > b, програмата разменя стойностите им, така че накрая a <= b.

```
#include <iostream>

int main() {
   int a, b;
   std::cout << "Enter a and b: ";
   if (!(std::cin >> a >> b)) {
      std::cout << "Invalid input.\n";
      return 1;
   }

   if (a > b) {
      int temp = a;
      a = b;
      b = temp;
   }

   std::cout << "a = " << a << ", b = " << b << '\n';
   return 0;
}
```

Примерно изпълнение:

```
Enter a and b: 6 2
a = 2, b = 6
```

3. Оператори if...else и else if

С конструкцията if...else се избира едно от две действия. При повече от две алтернативи се използва последователност else if; изпълнява се първият клон, чието условие е вярно.

```
if (условие1) {
   // първа възможност
} else if (условие2) {
   // втора възможност
} else {
   // останалите случаи
}
```

Пример 2. Определяне на годишен сезон

Въвежда се номер на месец от 1 до 12; програмата съобщава сезона или показва грешка при невалиден номер.

```
#include <iostream>

int main() {
   int month;
   std::cout << "Enter month number (1-12): ";
   if (!(std::cin >> month)) return 1;

   if (month == 12 || month == 1 || month == 2) {
      std::cout << "Winter\n";
   } else if (month >= 3 && month <= 5) {
      std::cout << "Spring\n";
   } else if (month >= 6 && month <= 8) {
      std::cout << "Summer\n";
   } else if (month >= 9 && month <= 11) {
      std::cout << "Autumn\n";
   } else {
      std::cout << "Invalid month.\n";
   }
   return 0;
}
```

Примерно изпълнение:

```
Enter month number (1-12): 3
Spring
```

Пример 3. Оценка по изпитни точки

Въведените точки са цяло число в интервала 0–100. Те се преобразуват до символна оценка по скалата: 90–100 → A, 80–89 → B, 70–79 → C, 65–69 → D, 0–64 → F.

```
#include <iostream>
#include <string>

int main() {
   int points;
   std::cout << " Enter points (0-100):";
   if (!(std::cin >> points)) return 1;

   if (points < 0 || points > 100) {
      std::cout << " Invalid number of points.\n";
      return 1;
   }

   std::string grade;
   if (points >= 89) grade = "Excellent (6)";
   else if (points >= 75) grade = "Very Good (5) ";
   else if (points >= 62) grade = "Good (4)";
   else if (points >= 50) grade = "Average (3)";
   else grade = "Poor (2)";

   std::cout << points << " points: " << grade << '\n';
   return 0;
}
```

Примерно изпълнение:

```
Enter points (0-100): 66
66 points: Good (4)
```

> Проверявайте по-общите и по-специалните условия в подходящ ред. Например проверката points >= 90 трябва да е преди points >= 80.

4. Оператор switch: множествен избор по стойност

switch е удобен, когато една целочислена, символна или изброена (enum) стойност се сравнява с фиксирани константи. След case може да има няколко оператора; break прекратява изпълнението на текущия switch. default обработва останалите стойности.

```
switch (израз) {
   case константа1:
      // действия
      break;
   case константа2:
      // други действия
      break;
   default:
      // няма съвпадение
}
```

Константите след case трябва да са уникални. switch не е заместител на условия като x > 10; за интервали обикновено е по-подходящ if...else.

Пример 4. Сезон по номер на месец със switch

```
#include <iostream>

int main() {
   int month;
   std::cout << "Enter month number (1-12): ";
   if (!(std::cin >> month)) return 1;

   switch (month) {
      case 12: case 1: case 2:
         std::cout << "Winter\n"; break;
      case 3: case 4: case 5:
         std::cout << "Spring\n"; break;
```

```
        case 6: case 7: case 8:
            std::cout << "Summer\n"; break;
        case 9: case 10: case 11:
            std::cout << "Autumn\n"; break;
        default:
            std::cout << "Invalid month.\n";
    }
    return 0;
}
```

Примерно изпълнение:

```
Enter month number (1-12): 8
Summer
```

5. Проверка на входните данни

В реални задачи проверката на данните е важна част от разклоненията. Ако входът трябва да се повтори до получаване на допустима стойност, може да се използва цикъл do...while.

Пример 5. Корени на квадратно уравнение

За уравнение $a \cdot x^2 + b \cdot x + c = 0$ коефициентът a трябва да е различен от 0. Дискриминантата $D = b^2 - 4ac$ определя броя на реалните корени: D < 0 — няма; D = 0 — един двоен; D > 0 — два.

```
#include <cmath>
#include <iostream>

int main() {
    int a, b, c;
    do {
        std::cout << "Enter a (nonzero): ";
        if (!(std::cin >> a)) return 1;
        if (a == 0) std::cout << "a cannot be zero.\n";
    } while (a == 0);

    std::cout << "Enter b and c: ";
    if (!(std::cin >> b >> c)) return 1;

    long long D = 1LL * b * b - 4LL * a * c;
    if (D < 0) {
        std::cout << "No real roots.\n";
    } else if (D == 0) {
        double x = -(double)b / (2.0 * a);
        std::cout << "Double root: x = " << x << '\n';
    } else {
        double x1 = (-(double)b + std::sqrt(D)) / (2.0 * a);
        double x2 = (-(double)b - std::sqrt(D)) / (2.0 * a);
        std::cout << "x1 = " << x1 << ", x2 = " << x2 << '\n';
    }
    return 0;
}
```

Примерно изпълнение:

```
Enter a (nonzero): 2
Enter b and c: 4 2
Double root: x = -1
```

6. Лабораторни задачи с разклонения и меню

Задача 4.1. „Мишка в лабиринт“

Лабиринтът има две врати X и Y. Потребителят въвежда за всяка Y/y (отворена) или N/n (затворена). Мишката може да излезе, ако поне една от вратите е отворена — X || Y.

```
#include <iostream>

int main() {
    char xAnswer, yAnswer;
    std::cout << "Is door X open (Y/N)? ";
```

```
    if (!(std::cin >> xAnswer)) return 1;
    std::cout << "Is door Y open (Y/N)? ";
    if (!(std::cin >> yAnswer)) return 1;

    bool xValid = xAnswer == 'Y' || xAnswer == 'y' ||
                  xAnswer == 'N' || xAnswer == 'n';
    bool yValid = yAnswer == 'Y' || yAnswer == 'y' ||
                  yAnswer == 'N' || yAnswer == 'n';
    if (!xValid || !yValid) {
        std::cout << "Enter Y or N only.\n";
        return 1;
    }
    bool X = xAnswer == 'Y' || xAnswer == 'y';
    bool Y = yAnswer == 'Y' || yAnswer == 'y';

    if (X || Y) std::cout << "The mouse can escape.\n";
    else std::cout << "The mouse cannot escape.\n";
    return 0;
}
```

Проверка: за N/N резултатът е „не може“, а за Y/N, N/Y и Y/Y — „може“.

Задача 4.2. Сравнение на тарифи на две транспортни фирми

Потребителят въвежда теглото на пратката weight (положително цяло число) и дали превозът е през нощта (Y/N). Програмата трябва да пресметне цените на Saturn и Jupiter и да предложи по-евтиния достъпен превозвач.

Saturn	Jupiter
До 30 kg: 10.00 лв.	През деня: 5.00 лв. + 0.10 лв./kg
31–70 kg: 20.00 лв.	През нощта: 7.00 лв. + 0.20 лв./kg
Над 70 kg: 0.50 лв./kg	Превозва пратки и през нощта
Не приема пратки над 70 kg през нощта	Няма ограничение в задачата

```
#include <iomanip>
#include <iostream>

int main() {
    int weight;
    char answer;
    std::cout << "Night delivery (Y/N)? ";
    if (!(std::cin >> answer)) return 1;
    std::cout << "Weight (kg): ";
    if (!(std::cin >> weight)) return 1;

    bool validAnswer = answer == 'Y' || answer == 'y' ||
                       answer == 'N' || answer == 'n';
    if (!validAnswer || weight <= 0) {
        std::cout << "Invalid input data.\n";
        return 1;
    }

    bool night = answer == 'Y' || answer == 'y';
    bool saturnAvailable = !(night && weight > 70);
    double saturnCost;
    if (weight <= 30) saturnCost = 10.0;
    else if (weight <= 70) saturnCost = 20.0;
    else saturnCost = 0.50 * weight;
    double jupiterCost = night ? 7.0 + 0.20 * weight
                               : 5.0 + 0.10 * weight;

    std::cout << std::fixed << std::setprecision(2);
    if (saturnAvailable) {
        std::cout << "Saturn: " << saturnCost << " BGN\n";
    } else {
        std::cout << "Saturn: no night delivery above 70 kg.\n";
```

```
    }
    std::cout << "Jupiter: " << jupiterCost << " BGN\n";

    if (!saturnAvailable || jupiterCost < saturnCost)
        std::cout << "Choose: Jupiter\n";
    else if (saturnCost < jupiterCost)
        std::cout << "Choose: Saturn\n";
    else
        std::cout << "Both companies offer the same price.\n";
    return 0;
}
```

Примерни проверки: 50 kg през деня → Jupiter 10.00 лв.; 29 kg през нощта → Saturn 10.00 лв.; 90 kg през нощта → само Jupiter.

Допълнителна задача: трети превозвач Venus

Разширете програмата с фирма Venus и два вида доставка. При пратки над 50 kg към основната цена се начислява цена само за килограмите над 50:

- Експресна доставка: 20.00 лв. до 50 kg; за всеки kg над 50 — още 0.50 лв.
- Обикновена доставка: 10.00 лв. до 50 kg; за всеки kg над 50 — още 0.25 лв.

Добавете избор на вид доставка и покажете тарифата на Venus. Сравнявайте услуги само при еднакви изисквания на клиента.

Задача 4.3. Изчисляване на заплата с меню и switch

От годишна заплата salary намерете стойността на едно плащане според payType: 0 — седмично (52 пъти годишно); 1 — два пъти месечно (24 пъти); 2 — веднъж месечно (12 пъти). Други стойности са невалидни.

```
#include <iomanip>
#include <iostream>

int main() {
    int payType;
    double salary;
    std::cout << "--- Payment schedule ---\n"
              << "0 - weekly (52 payments)\n"
              << "1 - twice a month (24 payments)\n"
              << "2 - monthly (12 payments)\n"
              << "Choose (0/1/2): ";
    if (!(std::cin >> payType)) return 1;
    if (payType < 0 || payType > 2) {
        std::cout << "Invalid choice.\n";
        return 1;
    }

    std::cout << "Annual salary (BGN): ";
    if (!(std::cin >> salary) || salary <= 0) {
        std::cout << "Invalid salary.\n";
        return 1;
    }

    int payments;
    switch (payType) {
        case 0: payments = 52; break;
        case 1: payments = 24; break;
        case 2: payments = 12; break;
        default: return 1;
    }
    std::cout << std::fixed << std::setprecision(2)
              << "Payment amount: " << salary / payments << " BGN\n";
    return 0;
}
```

При годишна заплата 30 000 лв. резултатите са: 0 → 576.92 лв.; 1 → 1250.00 лв.; 2 → 2500.00 лв.

Задача 4.4. Елементарен калкулатор

Потребителят въвежда израз, например 12*5 или 28/6. Обработете операциите +, -, *, / и % чрез switch. Операндите са цели числа; при / резултатът е дробно число, а % връща остатък от целочислено деление.

```
#include <iostream>

int main() {
   int x, y;
   char op;
   std::cout << "Enter expression (e.g. 12+3): ";
   if (!(std::cin >> x >> op >> y)) return 1;

   switch (op) {
      case '+': std::cout << x + y; break;
      case '-': std::cout << x - y; break;
      case '*': std::cout << x * y; break;
      case '/':
         if (y == 0) std::cout << "Division by zero!";
         else std::cout << double(x) / y;
         break;
      case '%':
         if (y == 0) std::cout << "Division by zero!";
         else std::cout << x % y;
         break;
      default: std::cout << "Invalid operator.";
   }
   std::cout << '\n';
   return 0;
}
```

Примерно изпълнение:

```
Enter expression (e.g. 12+3): 123*3
369
```

Задача 4.5. Определяне на ден от седмицата

Въведете дата във формат месец ден година, например 3 3 1985. За дати след 1752 г. изчислете номера на деня от седмицата по дадената формула и го преобразувайте чрез switch (0 — неделя, ..., 6 — събота).

За януари и февруари използвайте съответно месец 13 и 14 от предходната година. Целочислените деления във формулата отхвърлят дробната част:

```
calc_day = day + 2 * m + 3 * (m + 1) / 5 + y
         + y / 4 - y / 100 + y / 400 + 1;
weekday = calc_day % 7;
```

Преди изчислението проверете валидността на деня и месеца, включително високосните години.

```
#include <iostream>

int main() {
   int month, day, year;
   std::cout << "Enter date (mm dd yyyy): ";
   if (!(std::cin >> month >> day >> year)) return 1;

   if (year <= 1752 || month < 1 || month > 12) {
      std::cout << "Invalid date.\n";
      return 1;
   }
   bool leap = (year % 4 == 0 && year % 100 != 0) ||
               (year % 400 == 0);
   int days = 31;
   switch (month) {
      case 4: case 6: case 9: case 11: days = 30; break;
      case 2: days = leap ? 29 : 28; break;
   }
   if (day < 1 || day > days) {
      std::cout << "Invalid date.\n";
      return 1;
```

```
    }

    int m = month, y = year;
    if (m < 3) { m += 12; --y; }
    int calc_day = day + 2 * m + 3 * (m + 1) / 5 + y
                + y / 4 - y / 100 + y / 400 + 1;
    int weekday = calc_day % 7;

    std::cout << "Day of the week: ";
    switch (weekday) {
       case 0: std::cout << "Sunday"; break;
       case 1: std::cout << "Monday"; break;
       case 2: std::cout << "Tuesday"; break;
       case 3: std::cout << "Wednesday"; break;
       case 4: std::cout << "Thursday"; break;
       case 5: std::cout << "Friday"; break;
       case 6: std::cout << "Saturday"; break;
    }
    std::cout << '\n';
    return 0;
}
```

Примерно изпълнение:

```
Enter date (mm dd yyyy): 3 3 1985
Day of the week: Sunday
```

За тази задача се използва григорианското правило за високосна година. Ограничението за годината е част от условието на упражнението.

7. Въпроси и задачи за самостоятелна работа

1. Какъв е резултатът от логически израз в C++? Как се интерпретира 0 в условие?

2. Може ли логически израз да съдържа аритметични изрази? Дайте пример с && и ||.

3. Каква е разликата между if, if...else, else if и switch? Кога е подходяща всяка конструкция?

4. Може ли условието в if да е константа, променлива или израз? Какво ще стане при if (0) и if (1)?

5. Какви типове могат да се използват в switch? Защо не използваме case x > 10?

6. Каква е ролята на break и default? Защо стойностите след case не могат да се повтарят?

7. Група от X туристи посещава обект с входна такса Y лв. на човек. Посетителите под 18 години и студентите получават 40% отстъпка (при едновременно изпълнение — една отстъпка). Пресметнете общата сума, като проверите входните данни.

8. Търговски пътник има основна месечна заплата Y лв. и премия X лв. Пълна премия се получава при продажби поне 1500 лв. ИЛИ поне 30 нови клиенти. Премия 75% от X се получава, ако първото условие не е изпълнено, но продажбите са поне 500 лв. И новите клиенти са над 25. В останалите случаи премията е 0. Изчислете крайната заплата.

9. Въведете изпитните оценки за семестъра и изведете средния успех. При среден успех поне 5.50 стипендията е MAX лв.; от 4.50 до под 5.50 — 75% от MAX; от 3.50 до под 4.50 — 60% от MAX; под 3.50 — няма стипендия. Проверете валидността на оценките и MAX.

10. Учебна система за санкции: при превишение до 20 km/h глобата е 15% от зададена базова стойност MIN; от 21 до 40 — 25%; от 41 до 50 — 40%; от 51 до 70 — 70%; над 70 — съобщение за отнемане на правото на управление. Ако няма превишение — няма санкция. Въведете допустима и измерена скорост. Това са примерни учебни тарифи, а не действаща нормативна уредба.

Насоки за тестване

- Изпробвайте обичайни стойности, невалиден вход и гранични случаи (напр. 0, 30, 31, 70, 71).
- За всяка задача обяснете кое условие се е изпълнило и защо.
- Компилирайте и изпълнете решенията; проверете дали всички разклонения дават очакван резултат.